

A MemComputing Approach to Orbital Mechanics

Dietmar Henrich¹ and Massimiliano Di Ventra²

*¹⁾Faculty of Engineering and Computer Science, Brandenburg
University of Technology, Cottbus-Senftenberg, 01968 Senftenberg,
Germany*

*²⁾Department of Physics, University of California San Diego, La Jolla, CA 92093,
USA*

The MemComputing paradigm of computation employs time non-locality (memory) to solve a given problem by mapping its solutions into equilibrium points of a dynamical system. The problem is then solved not in the original state space of variables, but in an augmented phase space of relaxed original variables plus memory degrees of freedom. Although such an approach has been primarily used to solve combinatorial optimization problems, here we show that it can also be employed in the field of celestial mechanics. As an important example we introduce a MemComputing machine that discovers all five Lagrange equilibrium points of any restricted three-body system in the solar system, with mass ratios spanning eight orders of magnitude, without prior knowledge of their location. The dynamical system follows equations of motion in the co-rotating frame where memory degrees of freedom dynamically learn and modify the local character of the phase space landscape guiding the system towards the equilibria. We finally discuss this approach vis-à-vis existing methods.

I. INTRODUCTION AND SCOPE

MemComputing is a non-von-Neumann paradigm of computation in which a physical system exploits *time non-locality* (memory) to solve a given problem[?]. Here, memory is not synonymous with storage, rather it expresses the fact that the system retains information about its past, and exploits such information to perform useful computation. In this paradigm, the solutions of the target problem are mapped onto the stable fixed points of a continuous dissipative dynamical system, so that trajectories launched from generic initial conditions are guided to the solution along specific paths[?]. The computation proceeds in an augmented phase space whose coordinates are the relaxed original variables *plus* the memory degrees of freedom. To date this paradigm has been applied mostly to combinatorial optimization where input and output are binary[?]. However, the approach can be extended and applied to other equally important computational problems.

Here, we show that MemComputing can be easily adapted to solve an important continuous problem of celestial mechanics: the restricted three-body problem[?]. The restricted three-body problem (RTBP) consists in two massive primaries on a circular orbit and a massless test particle. It is a cornerstone of nonlinear dynamics and of practical importance for space mission design[?]. Its five equilibria, known as the Lagrange points[?], are normally located by analytic or numerical root finding that require a separate bracket or initial guess for each point, and prior knowledge of the solution structure.

In this paper, we construct instead a single MemComputing machine that discovers all five Lagrange points of *any* restricted three-body system in the solar system from a generic grid of initial conditions, without being given the coordinates of any solution. The machine is an autonomous dissipative dynamical system in the co-rotating frame with memory degrees of freedom that keep track of the local curvature of the phase space and *dynamically* transform saddle points of the original problem into minima of the expanded phase space, while leaving the original minima undisturbed. This way, the machine is always guaranteed to converge to one of the Lagrange points. We verify it on all twenty-three Sun–planet and planet–moon pairs, spanning mass ratios from $\mu \approx 2 \times 10^{-9}$ to $\mu \approx 0.1$, list every parameter needed to reproduce the results, and close by comparing the approach with existing methods. It is our hope that this example will inspire the community to employ the MemComputing paradigm in solving other celestial mechanics problems.

II. THE RESTRICTED THREE-BODY PROBLEM

As it is commonly done^{??}, we use the co-rotating frame in normalized units: the two primaries of masses M_1 and M_2 are separated by unit distance, the orbital angular velocity of the frame is $\omega = 1$, and the mass ratio is

$$\mu = \frac{M_2}{M_1 + M_2} \in (0, \frac{1}{2}], \quad (1)$$

placing M_1 at $(-\mu, 0)$ and M_2 at $(1 - \mu, 0)$. The Jacobi effective potential experienced by the massless test particle at position (x, y) is the sum of the gravitational potentials of the two primary masses plus the centrifugal potential within the rotating reference frame:

$$\Omega(x, y) = \frac{x^2 + y^2}{2} + \frac{1 - \mu}{r_1} + \frac{\mu}{r_2}, \quad (2)$$

with $r_1 = \sqrt{(x + \mu)^2 + y^2}$ and $r_2 = \sqrt{(x - 1 + \mu)^2 + y^2}$. This is the negative of the Jacobi potential commonly written with overall minus signs. With the convention (??) the Lagrange points are the critical points of Ω , namely those points in space at which the gradient of Ω evaluates to zero: $\nabla\Omega(x, y) = (0, 0)$.

From Eq. (??) the gradient is

$$\partial_x \Omega = x - \frac{(1 - \mu)(x + \mu)}{r_1^3} - \frac{\mu(x - 1 + \mu)}{r_2^3}, \quad (3)$$

$$\partial_y \Omega = y - \frac{(1 - \mu)y}{r_1^3} - \frac{\mu y}{r_2^3}. \quad (4)$$

The critical point condition then provides exactly five solutions: the collinear points L_1, L_2, L_3 (which do not have a closed-form analytical expression) on the axis $y = 0$ and the triangular points $L_{4,5} = (\frac{1}{2} - \mu, \pm \frac{\sqrt{3}}{2})$ ^{??}.

The *nature* of each critical point—which is what the machine must contend with—is set by the Hessian of Ω . Its transverse (y) component is

$$\Omega_{yy} = 1 - \frac{1 - \mu}{r_1^3} + \frac{3(1 - \mu)y^2}{r_1^5} - \frac{\mu}{r_2^3} + \frac{3\mu y^2}{r_2^5}. \quad (5)$$

On the axis ($y = 0$) this reduces to $1 - (1 - \mu)/r_1^3 - \mu/r_2^3$, which is *negative* at all three collinear points. Together with the fact that the x component of the Hessian $\Omega_{xx} = 1 +$

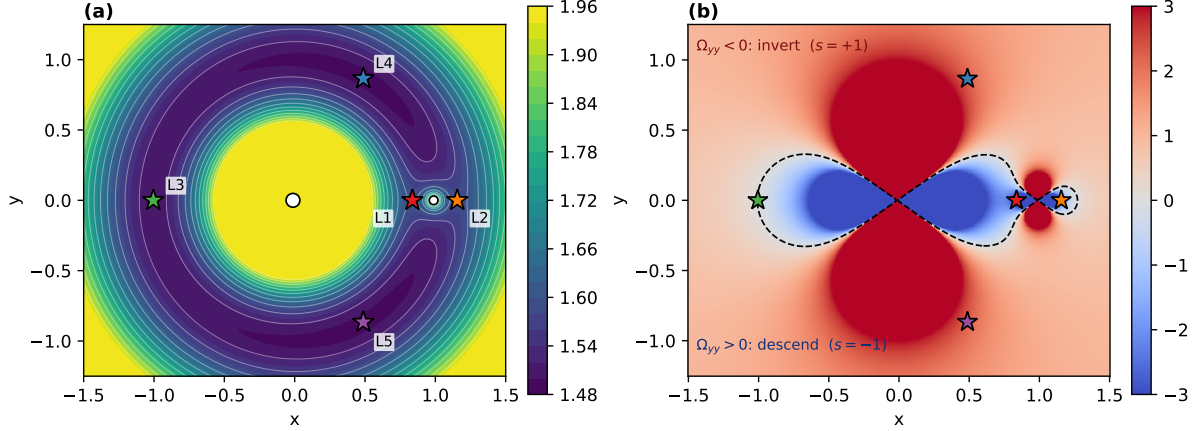


FIG. 1. The effective potential and its curvature for the Earth–Moon system ($\mu = 0.0121$). **(a)** $\Omega(x, y)$, Eq. (??); the collinear L_1, L_2, L_3 are saddles and the triangular L_4, L_5 are minima; the large and small white dots are the primary and secondary masses, respectively. **(b)** the transverse curvature Ω_{yy} , Eq. (??) (clipped at ± 3), with the contour $\Omega_{yy} = 0$ dashed. Where $\Omega_{yy} < 0$ (red) the MemComputing dynamics inverts the transverse force ($s = +1$) to make the saddle attract; where $\Omega_{yy} > 0$ (blue) it descends normally ($s = -1$). The machine never reads this map; it learns the sign from its own gradient history.

$2(1 - \mu)/r_1^3 + 2\mu/r_2^3 > 0$ and $\Omega_{xy} = 0$ there, the collinear points are *saddles*, unstable in the transverse (y) direction. At the triangular points one finds $\Omega_{xx} = \frac{3}{4}$, $\Omega_{yy} = \frac{9}{4}$, and $\Omega_{xy} = \frac{3\sqrt{3}}{4}(1 - 2\mu)$, giving $\det[\text{Hess } \Omega] = \frac{27}{4}\mu(1 - \mu) > 0$ with positive trace: L_4, L_5 are *minima* of Ω . The sign of Ω_{yy} thus distinguishes the two cases: it is negative in the unstable direction of the collinear saddles and positive at the triangular minima (Fig. ??).

The presence of saddle points means that a dynamical system approaching them would be repelled by the unstable manifold and unable to really find their position. What is then needed is a machine that “learns” the curvature of the Jacobi potential and smoothly converts the saddle points into equilibria, while keeping the equilibria unaffected. All this using only information generated along its own trajectory. This is the essence of the MemComputing approach where the solutions are then the stable fixed points of the autonomous flow in the augmented space of the original variables plus memory degrees of freedom.

III. MEMORY-INDUCED PHASE-SPACE ENGINEERING

The equations of motion of this MemComputing machine can be easily determined. To the ordinary rotating-frame equations of motion—the gradient force $-\nabla\Omega$ and the Coriolis acceleration—we add a viscous term (to make the dynamical system dissipative, hence supporting attractors) and introduce a memory degree of freedom, B , which learns the local curvature of the Jacobi potential, and feeds into a second one, s , whose role is to change the index of the critical points accordingly. More precisely, s is tasked to change the collinear saddle points to minima, while leaving the character of the triangular points untouched.

The damped flow that descends Ω in the rotating frame is then,

$$\ddot{x} = 2\dot{y} - \partial_x\Omega - \gamma\dot{x}, \quad (6a)$$

$$\ddot{y} = -2\dot{x} + s\partial_y\Omega - \gamma\dot{y}, \quad (6b)$$

with constant viscosity $\gamma > 0$ and the memory term s multiplying the transverse force. We will solve these equations using the simple forward Euler with a fixed time step[?].

Whenever s approaches -1 the transverse force is the ordinary descent term $-\partial_y\Omega$, which is restoring at a minimum. The flow then settles onto the minima of Ω , i.e., the triangular points L_4, L_5 . At a collinear saddle, however, the transverse direction has $\Omega_{yy} < 0$: a small displacement $y > 0$ gives $\partial_y\Omega < 0$, so it pushes the particle *away* from the axis and the saddle repels. Having the memory approaching $s = +1$ reverses this single term into a restoring force, converting the saddle’s unstable direction into a stable one—the saddle becomes a minimum of the effective, inverted landscape and an attractor of the flow. The key point is that this memory term is intrinsically position-dependent, and no constant value of s would work in the whole phase space.

IV. LEARNING THE CURVATURE

The adaptive rule of Sec. ?? requires the sign of Ω_{yy} at every point in space. Although reading it from the analytic Hessian (??) would work, this route would be unsatisfying: the machine would be handed the very curvature information that distinguishes the solutions. We instead let a memory degree of freedom *learn* it. The flow already evaluates the gradient

$\mathbf{g} = \nabla\Omega$ at every step of the computation, and the Jacobian of $\mathbf{g}$ is the Hessian. A memory matrix B therefore accumulates a *quasi-Newton* estimate of the Hessian from successive gradient samples. Setting $\Delta\mathbf{p} = \mathbf{p}_k - \mathbf{p}_{k-1}$, $\Delta\mathbf{g} = \mathbf{g}_k - \mathbf{g}_{k-1}$, and $\Delta t = t_k - t_{k-1}$, at steps k and $k-1$ of the simulation, we can then update the matrix B from step k to step $k+1$ (call these matrices B_k and B_{k+1} , respectively) with a Broyden (quasi-Newton) secant update[?] [?] and a slow relaxation term toward the identity,

$$B_{k+1} = B_k + \frac{(\Delta\mathbf{g} - B_k \Delta\mathbf{p}) \Delta\mathbf{p}^\top}{\Delta\mathbf{p}^\top \Delta\mathbf{p}} + \lambda (\mathbf{1} - B_k) \Delta t, \quad (7)$$

applied only whenever $\|\Delta\mathbf{p}\|$ exceeds a small threshold (otherwise $B_{k+1} = B_k$). The result is then symmetrized at every step, by performing the operation $(B_{k+1} + B_{k+1}^\top)/2$, to respect the symmetry of a Hessian. The two added terms in Eq. (??) are of different character. The first is the secant correction, which enforces $B \Delta\mathbf{p} = \Delta\mathbf{g}$ along the current step: a full per-step assimilation whose size is set by how wrong B is and is *independent* of Δt (Broyden is a discrete iteration, with no continuum limit as Δt shrinks to zero). The second is a genuine rate—a slow leak toward the identity. Because the curvature varies across the landscape, this leak gives B a finite memory horizon $\sim 1/\lambda$, so it tracks the *local* curvature rather than a path-history average, and relaxing toward $\mathbf{1}$ supplies a safe default ($B_{yy} = 1$) where the step is too small to measure. Equivalently, B follows a continuous leak $\dot{B} = -\lambda(B - \mathbf{1})$ punctuated by discrete secant measurements—the structure of a continuous–discrete (Kalman-type) estimator[?]. The estimated transverse curvature is the component B_{yy} , that enters the memory variable $s \in [-1, 1]$ which relaxes toward the learned sign,

$$\dot{s} = -\alpha \left[\tanh\left(\frac{B_{yy} - \theta_s}{\varepsilon_c}\right) + s \right]. \quad (8)$$

Here, the threshold $\theta_s < 0$ in (??) biases s toward the safe descending state $s = -1$, inverting only when the learned curvature is *clearly* negative. This protects the minima and the small- μ corotation ridge (where $\|\nabla\Omega\| \approx 0$ along $r = 1$ and the curvature estimate hovers near zero) while still inverting at genuine saddles, where B_{yy} is strongly negative. The bias is needed because the damped rotating frame is not symmetric under $y \rightarrow -y$ —the Coriolis term breaks it—so L_4 and L_5 have slightly different basins. Therefore, biasing toward the safe state treats both robustly.

TABLE I. Parameters used for the MemComputing dynamics. They are the same for all twenty-three Sun–planet and planet–moon pairs.

Symbol	Meaning	Value
γ	viscosity (constant)	0.3
α	relaxation rate of the memory s	15
ε_c	curvature scale in Eq. (??)	0.3
λ	Broyden leak toward $\mathbf{1}$	2.0
θ_s	saddle to minima if $B_{yy} < \theta_s$	−0.6
$\ \Delta\mathbf{p}\ _{\min}$	Broyden update gate	10^{-4}
Δt	Euler time step	0.01
θ	convergence threshold $\ \nabla\Omega\ $	10^{-4}
$N_{\max}$	maximum steps	1.2×10^5
ε	singularity threshold on r_1, r_2	10^{-9}

The parameter ε_c controls the width of the window around θ_s , namely how sharply B_{yy} crossing the threshold triggers a change of index of the saddle points. Finally, α controls how fast the memory variable s varies. It is chosen such that the relaxation time of s , $\sim 1/\alpha$ is short compared to saddle’s instability time scale (of order one in the rotating-frame units).

The augmented phase space is thus $(x, y, \dot{x}, \dot{y}, s, B)$: the mechanical variables plus the memories (s, B) . The full machine integrates (??) with γ constant, advancing B by (??) and s by (??) at each step. No analytic second derivative appears anywhere in the dynamics (Ω_{yy} may be used only, after the fact, to confirm that the learned B_{yy} is correct).

We note that the resulting scheme is *predictive*. During the off-axis approach to a saddle the trajectory samples gradients over a range of positions, so B accumulates a usable curvature estimate *before* the trajectory reaches the unstable region—it does not have to wait for the runaway to develop. A purely reactive scheme, namely one that would infer instability from the growth of the motion, would be outrun by the exponential instability of the saddle and fail. The quasi-Newton accumulation is what makes the memory fast enough.

V. NUMERICAL IMPLEMENTATION AND RESULTS

We now provide details of the numerical implementation. The parameters used for the MemComputing dynamics are collected in Table ?? . A demo code of the simulations can found at[?] .

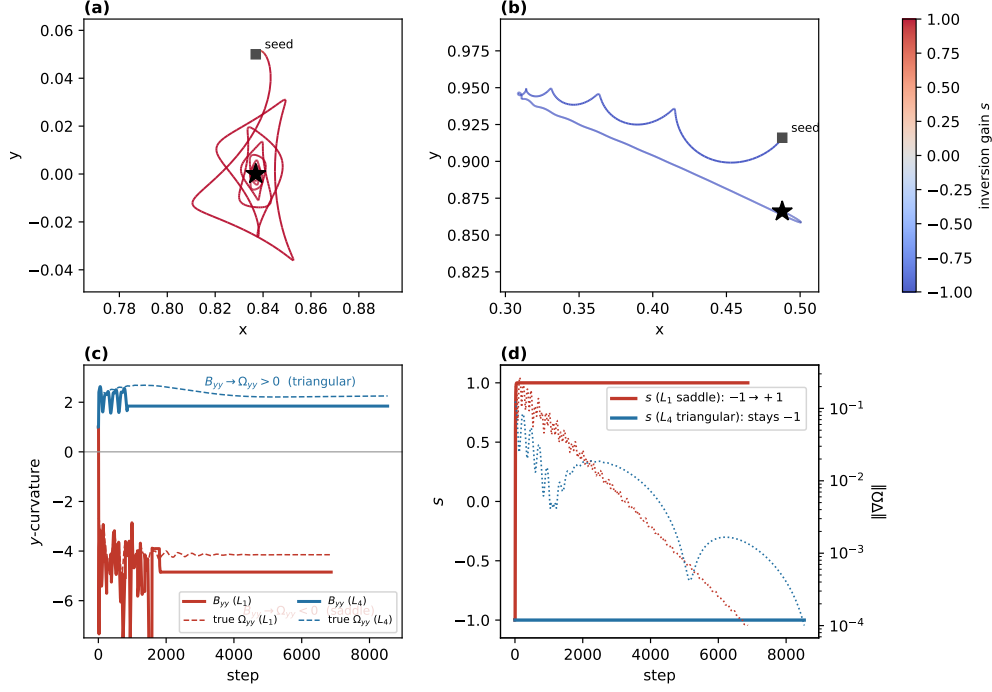


FIG. 2. Two representative Earth–Moon trajectories of the MemComputing dynamics. (a),(b) phase-plane paths colored by the value of the memory variable s : red ($s \rightarrow +1$) on the approach to the collinear saddle L_1 (black star), blue ($s \rightarrow -1$) on the approach to the triangular minimum L_4 (black star). (c) the Broyden estimate B_{yy} (solid) tracks the true Ω_{yy} (dashed) from gradient history alone—negative at the saddle, positive at the minimum. (d) the memory s responds to what was learned: $-1 \rightarrow +1$ at the saddle, held at -1 at the minimum, while $\|\nabla\Omega\|$ (dotted lines) decays to convergence.

Integration. As anticipated, the equations of motion are integrated by explicit forward Euler⁷ with fixed step Δt . A threshold $\varepsilon = 10^{-9}$ is added to r_1, r_2 to avoid divergences. The memory is initialized at $B = \mathbf{1}$, $s = -1$ (the safe descending state), and the Broyden update (??) is applied only when $\|\Delta\mathbf{p}\|$ exceeds a small threshold, so that near a fixed point—where $\Delta\mathbf{p} \rightarrow 0$ —the estimate simply stops. A trajectory ends when the magnitude of the gradient is below a threshold ($\|\nabla\Omega\| < \theta$), when it leaves the disk $\|\mathbf{r}\| > 12$, or after $N_{\max}$ steps.

Initial conditions grid. No solution coordinates are given to the dynamics. Starting positions are generated from the *generic structure* of the RTBP only, using the Hill radius $r_H = (\mu/3)^{1/3}$ and the secondary position $x_s = 1 - \mu$ as follows: (i) on-axis seeds $(x_s \pm c r_H, 0)$ for $c \in \{0.5, 0.7, 1, 1.4, 2, 3, 5, 8\}$, bracketing the collinear L_1, L_2 ; (ii) L_3 seeds near $x = -1$; (iii) equilateral seeds about $(\frac{1}{2}, \pm \frac{\sqrt{3}}{2})$ for the triangular points; (iv) a coarse off-axis fill for the broad large- μ basins. Initial conditions inside the exclusion disks around either primary

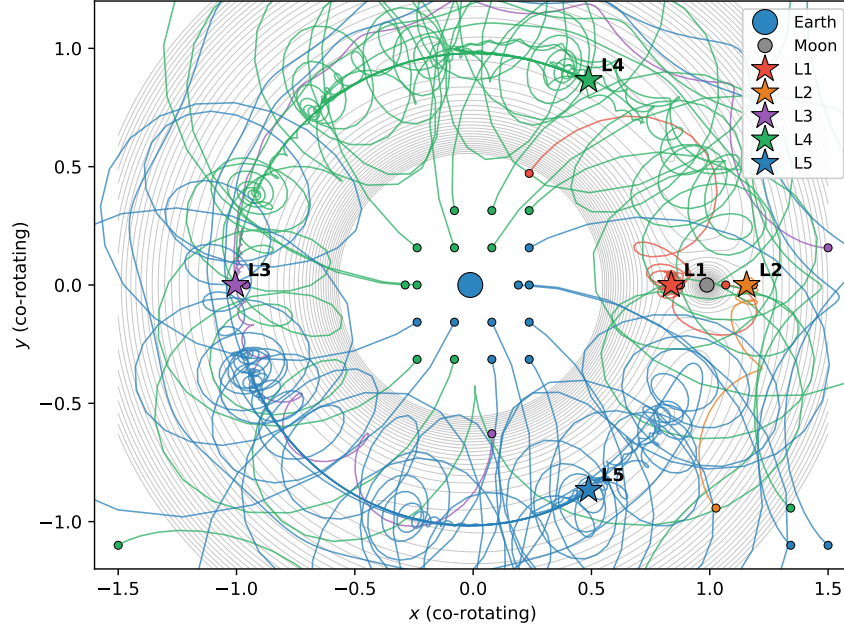


FIG. 3. Representative trajectories of the MemComputing dynamics for the Earth–Moon system ($\mu = 0.0121$), each colored by the Lagrange point it converges to (stars). Circles mark the initial conditions; grey curves are level sets of Ω ; the large and small filled circles are the primary (Earth) and secondary (Moon). The machine receives no solution coordinates and no analytic curvature.

are discarded, leaving of order seventy initial conditions per system. Even the equilateral initial conditions use the fixed $(\frac{1}{2}, \pm\frac{\sqrt{3}}{2})$ rather than the μ -dependent solution.

As an example, Figure ?? shows the memory at work on two representative Earth–Moon trajectories. On the approach to the collinear saddle L_1 the estimate B_{yy} converges to a negative value tracking the true Ω_{yy} , and the memory s ramps from -1 to $+1$, inverting the force and capturing the saddle. On the approach to the triangular minimum L_4 the estimate B_{yy} is positive, s stays at -1 , and the flow descends normally. The same memory, with no curvature supplied, discovers opposite signs in the two cases. In Figure ?? we show representative trajectories of the machine reaching all five points of the Earth–Moon system from five of the the initial condition points.

The central result is that the MemComputing dynamics, with the same parameter set of Table ?? and *no analytic Hessian in its dynamics*, discovers all five Lagrange points of every two-body system in the solar system. Table ?? lists the twenty-three Sun–planet and

TABLE II. All five Lagrange points found for every solar-system pair by the MemComputing dynamics (without any analytic Hessian), using the same parameters of Table ?? for all pairs. “Stable” refers to the linear stability of L_4, L_5 (Routh, $\mu < 0.03852^?$).

System	μ	Found L_4/L_5 stable	
Mars–Deimos	2.3×10^{-9}	5/5	yes
Sun–Pluto	6.6×10^{-9}	5/5	yes
Mars–Phobos	1.7×10^{-8}	5/5	yes
Sun–Mercury	1.7×10^{-7}	5/5	yes
Saturn–Enceladus	1.9×10^{-7}	5/5	yes
Sun–Mars	3.2×10^{-7}	5/5	yes
Sun–Venus	2.4×10^{-6}	5/5	yes
Sun–Earth	3.0×10^{-6}	5/5	yes
Saturn–Rhea	4.1×10^{-6}	5/5	yes
Jupiter–Europa	2.5×10^{-5}	5/5	yes
Uranus–Oberon	3.5×10^{-5}	5/5	yes
Uranus–Titania	4.1×10^{-5}	5/5	yes
Sun–Uranus	4.4×10^{-5}	5/5	yes
Jupiter–Io	4.7×10^{-5}	5/5	yes
Sun–Neptune	5.1×10^{-5}	5/5	yes
Jupiter–Callisto	5.7×10^{-5}	5/5	yes
Jupiter–Ganymede	7.8×10^{-5}	5/5	yes
Neptune–Triton	2.1×10^{-4}	5/5	yes
Saturn–Titan	2.4×10^{-4}	5/5	yes
Sun–Saturn	2.9×10^{-4}	5/5	yes
Sun–Jupiter	9.5×10^{-4}	5/5	yes
Earth–Moon	1.2×10^{-2}	5/5	yes
Pluto–Charon	1.1×10^{-1}	5/5	no

planet–moon pairs, with mass ratios spanning eight orders of magnitude, from Mars–Deimos ($\mu = 2.3 \times 10^{-9}$) to Pluto–Charon ($\mu = 0.11$). Every system returns 5/5 Lagrange points.

The smaller- μ systems are, in general, the most difficult ones to tackle—the triangular minima for these systems sit on the nearly flat co-rotation ridge—and they are exactly where the threshold θ_s matters the most. With it, the MemComputing dynamics adjusts to the correct curvature of the Jacobi potential across the whole range.

VI. ADVANTAGES AND LIMITATIONS OF THIS APPROACH

The MemComputing approach clearly finds all Lagrange points of every solar-system pair with just one set of parameters. However, if we could identify a suitable volume around a Lagrange point, classical root finders, like the Newton method[?], would converge towards

that point in a handful of evaluations, namely much faster than integrating a trajectory. This could be construed as a disadvantage of this approach. The advantage with respect to these traditional methods is then of a different kind. The machine is *global and prior-knowledge-free*: one autonomous flow, one parameter set, locates all five equilibria of any system from a generic grid, without a bracket per point and without being told which points are saddles. More broadly, it finds critical points of *any index*, not only minima—the force inversion turns a saddle into an attractor—which ordinary gradient descent and the optimization methods built on it cannot do.

Beyond Newton’s method, several other approaches are used in practice to locate Lagrange points. First, the triangular points L_4, L_5 are always at the exact closed-form positions $(\frac{1}{2} - \mu, \pm \frac{\sqrt{3}}{2})$, so no numerical step is needed. Second, the collinear points have no closed form but their locations can be expressed as power series in $\mu^{1/3}$; for small mass ratios these series are accurate enough to avoid further iteration. Third, since L_1, L_2, L_3 all lie on the x -axis ($y = 0$), a simple one-dimensional bisection on $\partial_x \Omega$ locates each one in a few function evaluations, because the gradient changes sign at every collinear point. Finally, homotopy (continuation) methods track each equilibrium as μ varies from a known value, following the solution branch smoothly to the target mass ratio. All these methods are faster per point than the MemComputing dynamics when they apply—but each requires either exact prior structural knowledge (the triangular closed form, the collinear axis constraint) or a separate bracket or continuation path per point. The MemComputing machine requires none of this: it discovers all five points simultaneously from a generic grid, treating saddles and minima on equal footing.

In the particular example of this paper, the inversion of the index of the critical point is a single transverse sign because the collinear saddles’ unstable direction is the y -axis. However, for a general situation, the same memory matrix B supplies the unstable eigendirections, and the force would be inverted along them. We thus expect this approach could be generalized to other types of problems in orbital mechanics, where finding saddle points of arbitrary symmetry may be necessary.

VII. CONCLUSION

In conclusion, we have introduced a MemComputing machine that locates all five Lagrange points of the restricted three-body problem by descending the Jacobi effective potential while inverting the force along the unstable direction of its saddles. The inversion sign cannot be constant—no fixed choice captures both the saddles and the minima—so it is carried by memory degrees of freedom, which learn the local curvature as a quasi-Newton estimate accumulated from the machine’s own gradient evaluations, with no analytic second derivative in the dynamics. The machine discovers all five points of every Sun–planet and planet–moon pair in the solar system—twenty-three systems with mass ratios ranging from $\mu \approx 2 \times 10^{-9}$ to 0.1—from a generic grid of initial conditions and a single set of parameters. This problem extends the applicability of the MemComputing paradigm to celestial mechanics and provides another example in which memory (time non-locality) aids in the computation. We therefore hope that this work will inspire the use of the MemComputing paradigm in other studies of celestial mechanics.

ACKNOWLEDGMENTS

The authors thank Prof. G. Cuniberti for the hospitality during the writing of this work. M. D. acknowledges support from the National Science Foundation via grant no. ECCS-2229880 and the Alexander von Humboldt Stiftung through the Humboldt Research Award.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request. A demo code to reproduce the simulations reported in this paper is available at[?].

REFERENCES

- ¹M. Di Ventra, *MemComputing: Fundamentals and Applications of Time Non-Locality*. Oxford University Press, 2022.

- ²M. Di Ventra and F. L. Traversa, “Perspective: Memcomputing: Leveraging memory and physics to compute efficiently,” *J. Appl. Phys.*, vol. 123, no. 18, p. 180901, 2018.
- ³S. R. B. Bearden, Y. R. Pei, and M. Di Ventra, “Efficient solution of Boolean satisfiability problems with digital MemComputing,” *Scientific Reports*, vol. 10, no. 1, p. 19741, 2020.
- ⁴F. L. Traversa and M. Di Ventra, “Polynomial-time solution of prime factorization and NP-complete problems with digital memcomputing machines,” *Chaos*, vol. 27, no. 2, p. 023107, 2017.
- ⁵V. Szebehely, *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, 1967.
- ⁶W. S. Koon and M. W. Lo and J. E. Marsden and S. D. Ross, *Dynamical Systems, the Three-Body Problem and Space Mission Design*. Springer, 2011.
- ⁷J.-L. Lagrange, “Essai sur le problème des trois corps,” *Prix de l’Acad. R. Sci. Paris*, vol. 9, 1772.
- ⁸E. J. Routh, “On Laplace’s three particles, with a supplement on the stability of steady motion,” *Proc. London Math. Soc.*, vol. s1-6, pp. 86–97, 1875.
- ⁹T. Sauer, *Numerical Analysis*. Pearson, New York, 2018.
- ¹⁰C. G. Broyden, “A class of methods for solving nonlinear simultaneous equations,” *Math. Comp.*, vol. 19, no. 92, pp. 577–593, 1965.
- ¹¹J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- ¹²D. Henrich, “MemComputing orbital mechanics—interactive demo,” <https://digitalmemcomputing.streamlit.app>, 2026.